



Exercise 17: Inertial Interface

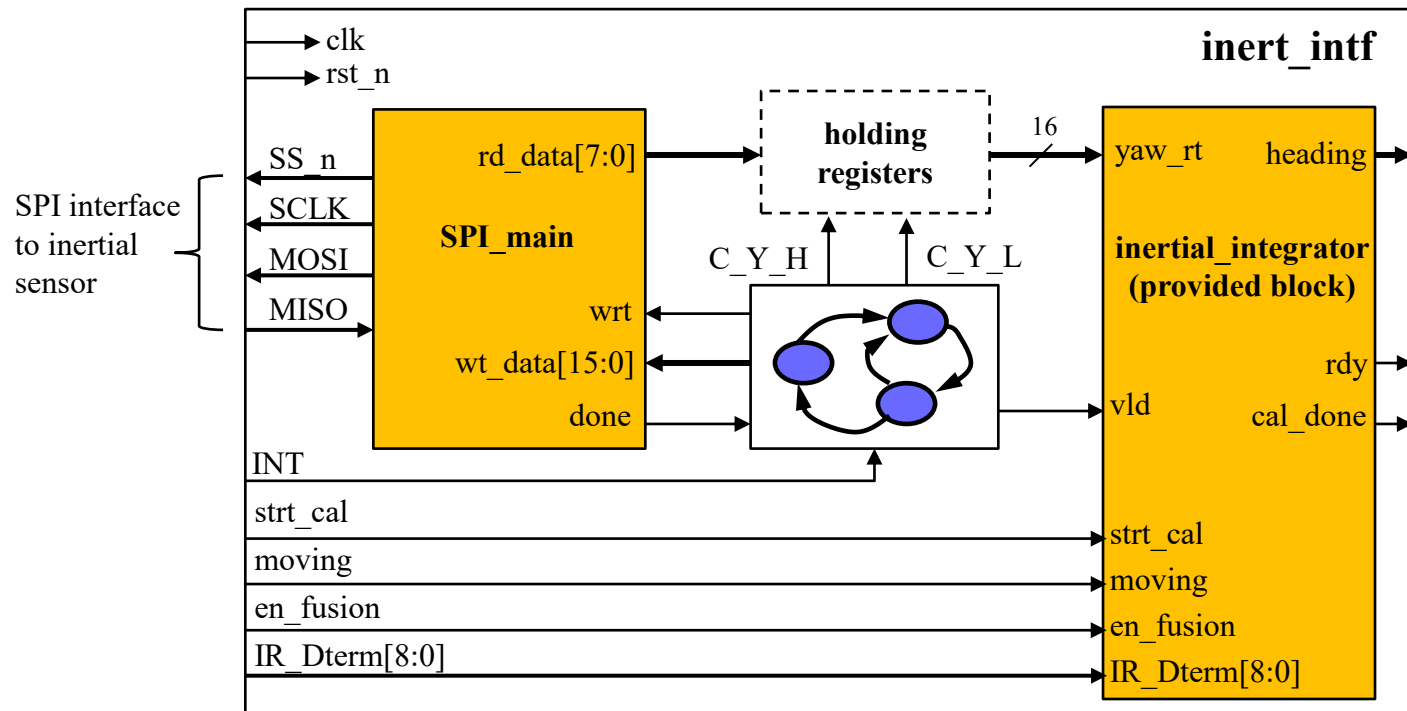
Break into two teams of 2 for this exercise

Each team of 2 should work on this separately so you have two choices for your inert_intf.sv design



Exercise 17: Inertial Interface

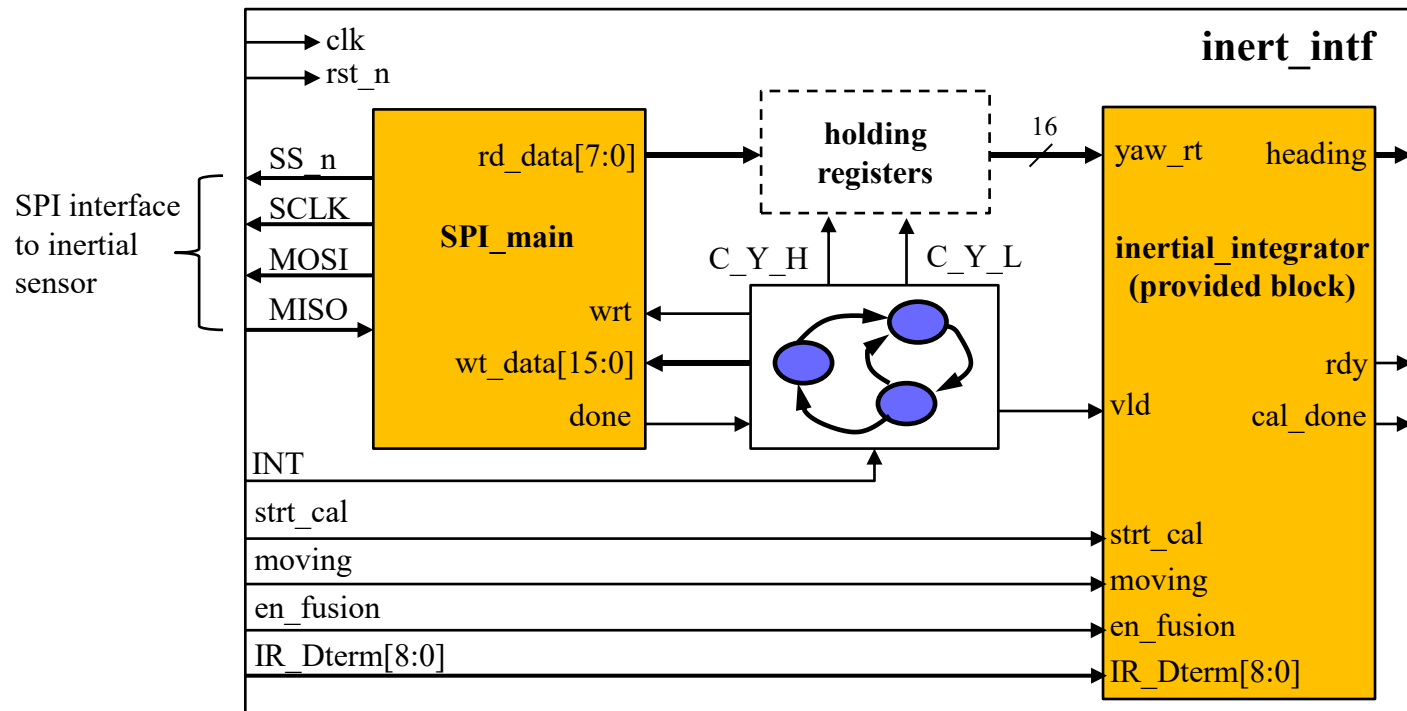
- The inertial interface will handle communication with the inertial sensor
- It needs to do a few things:
 - Configure the inertial sensors (INT, data rate, burst read)
 - Calibration
 - Read whenever a new reading is available
 - Convert from yaw rate to yaw & adjust based on IR sensor value





Exercise 17: Inertial Interface

- The inertial interface will handle communication with the inertial sensor
- It needs to do a few things:
 - Configure the inertial sensors (INT, data rate, burst read)
 - Calibration
 - Read whenever a new reading is available
 - Convert from yaw rate to yaw & adjust based on IR sensor value



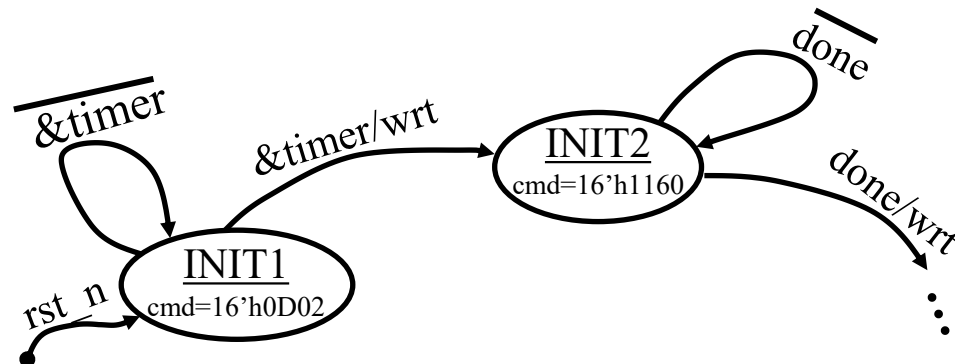


Configure Inertial Sensor

- After reset de-asserts, the system must configure the inertial sensor to the mode we wish to operate in
- The following table outlines writes you need to perform to set it up

Addr	Description
0x0D	Enable interrupt upon data ready by writing 0x02.
0x11	Setup gyro for 416Hz data rate, +/- 250°/sec range by writing 0x60.
0x14	Turn on wraparound reads for gyro by writing 0x40.

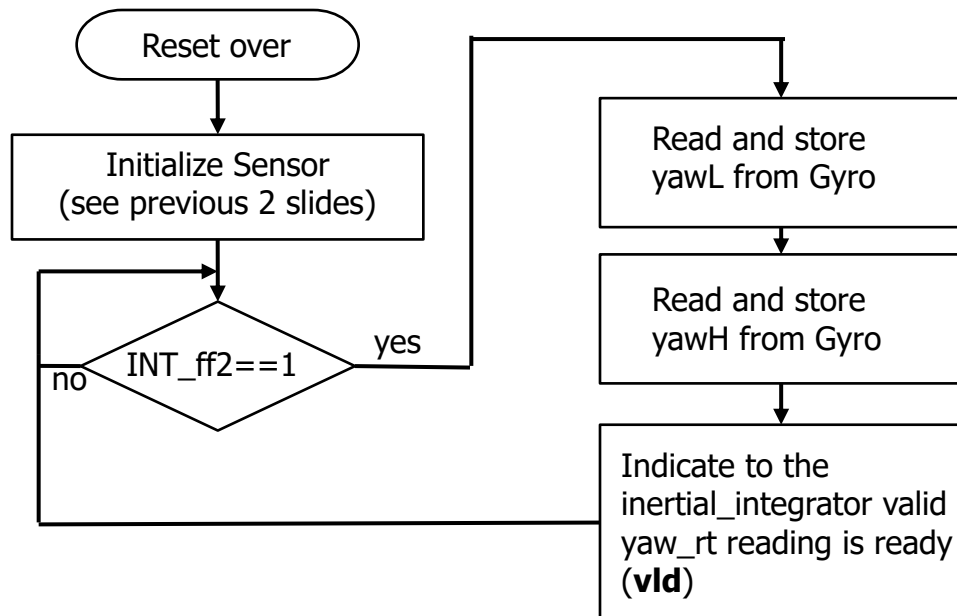
- Your FSM will need to have a few states dedicated to initialization
- The inertial sensor has a reset sequence that takes a while
→ use a 16-bit timer to create a delay before 1st write





Reading Sensor Values

- After configuring the inertial sensor, you should be in an infinite loop of reading gyro yaw data
- The sensor provides an active high **INT** when new data is ready
- Double flop it (for metastability reasons) and initiate a sequence of 2 reads from the gyro
- You should have two 8-bit FFs to store the 2 needed readings

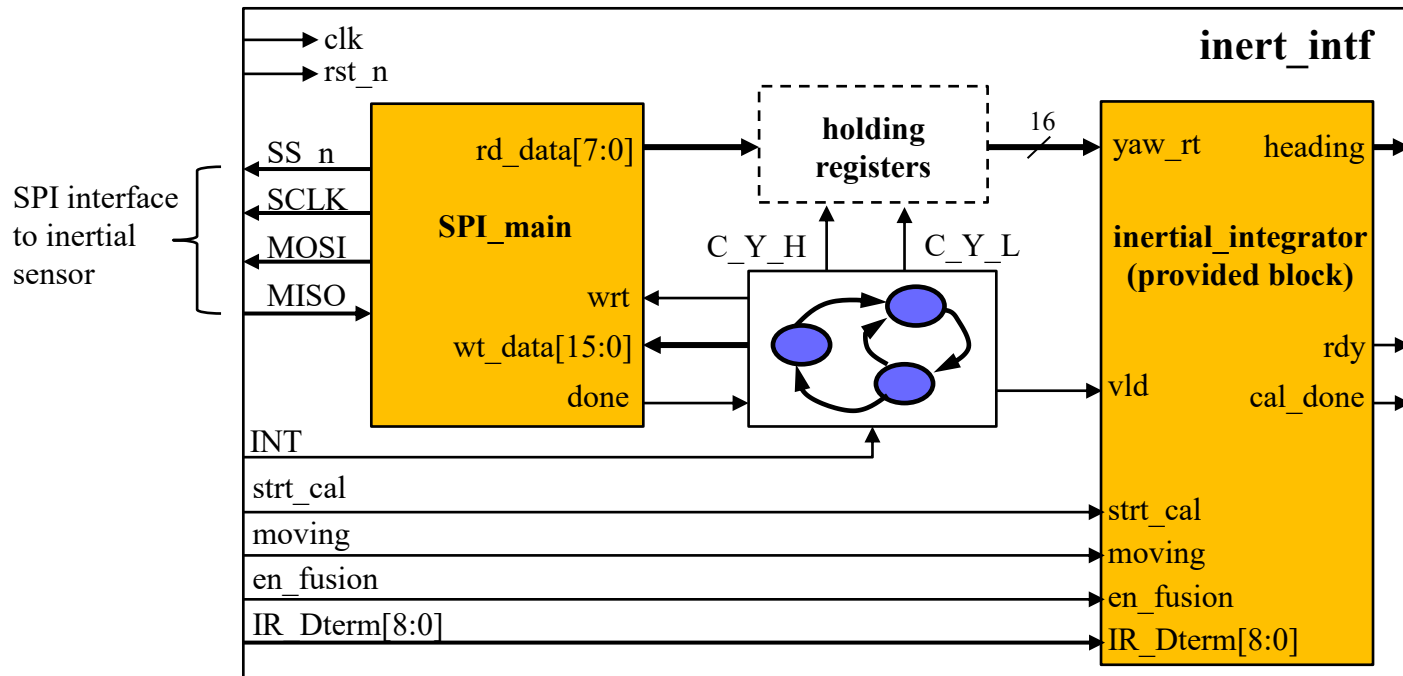


Addr	Description
7'h26	yawL → yaw rate low from gyro
7'h27	yawH → yaw rate high from gyro



Calibration

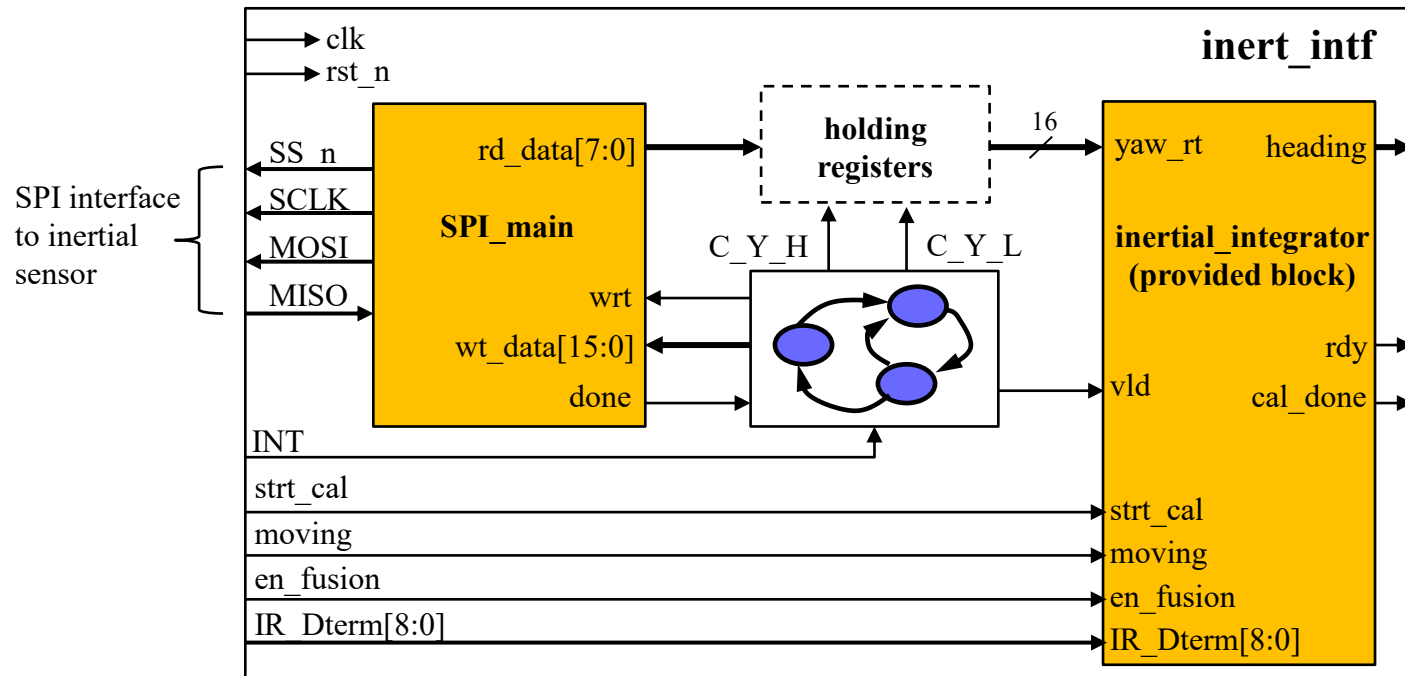
- MEMs gyro readings are in terms of yaw rate $\rightarrow$ integrate it to obtain yaw
 - Small errors will cause drift $\rightarrow$ we will take 2048 readings to determine an offset, then subtract it from subsequent readings
 - The MazeRunner can receive a **strt_cal** signal when it is not moving
 - When calibration is done, **cal_done** will be asserted
- $\rightarrow$ Computation is handled in the provided block, you need to ensure that you connect **strt_cal**, **moving**, and keep providing readings from gyro





Forming Heading

- Once calibration is completed, the MazeRunner will be integrating the yaw readings to provide the **heading** when it is **moving**
- When known to be moving forward (**en_fusion**), the **IR_Dterm[8:0]** will be used in a form of sensor fusion to correct drift in the gyro readings, in addition to the offset computed during calibration
→ You can take a look at how this is handled in inertial_integrator





Exercise 17: Inertial Interface

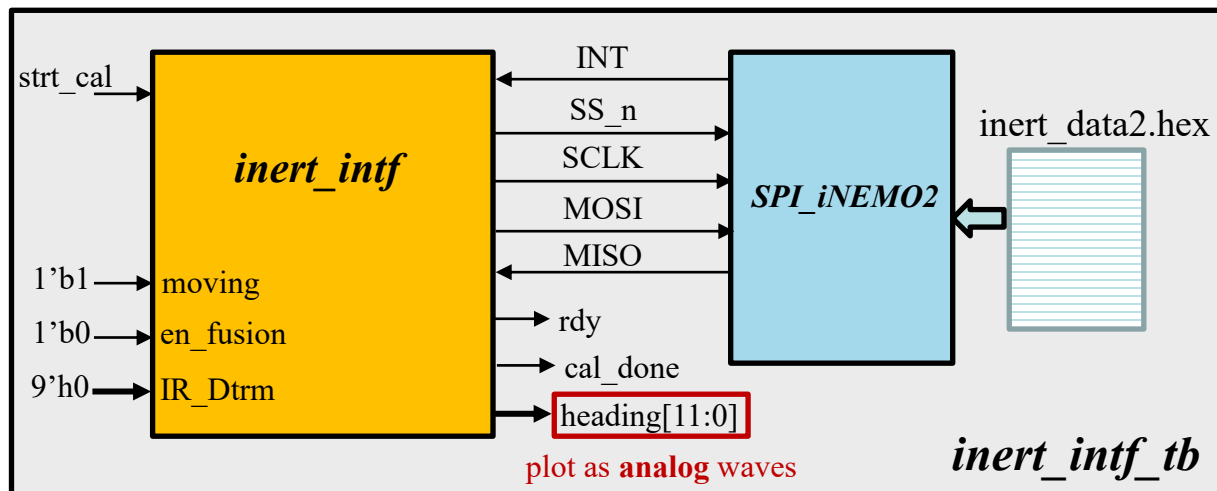
- A shell (**iner_intf_shell.sv**) is provided
- Rename to **inert_intf.sv** and complete it

Signal	Dir	Description
clk, rst_n	in	50MHz clock and active low asynch reset
strt_cal	in	From cmd_proc unit. Tells inertial_integrator to start cal.
INT	in	INT pin from inertial sensor, tells us a new measurement is ready. Is this synched to your clk domain?
cal_done	out	From inertial_integrator to cmd_proc . Asserted when calibration complete
rdy	out	Asserted from SM inside inert_intf . Consumed in inertial_integrator , but also an output to flight_cntrl .
heading[11:0]	out	This is the primary output of inert_intf . This is a signed number where 000 represents original direction, 3FF is 90° CCW, 7FF = 180° CCW, BFF = 90° CW...
moving	in	Integration of gyro yaw readings is inhibited when not moving
en_fusion	in	Enables fusion correction.
SS_n, SCLK, MOSI, MISO	out / in	SPI interface to inertial sensor



Testing Inertial Interface

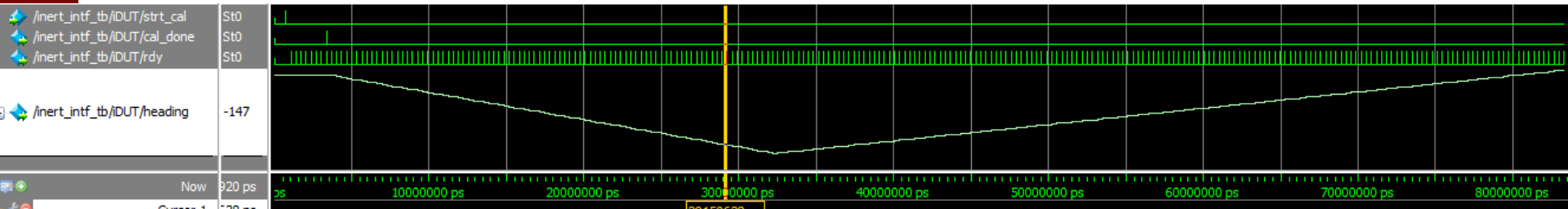
- A different model (**SPI_iNEMO2.sv** & **inert_data2.hex**) is provided
- Modify your DUT to take a **FAST_SIM** param to change calibration to only take 8 samples (vs 2048) for simulation
- Build your testbench as shown below & do the following to test:
 1. Reset the DUT
 2. Wait for **NEMO_setup** inside SPI_iNEMO2 to get asserted.
→ This is a good place to use the fork/join technique for waiting on a signal to assert, but having a timeout if it doesn't
 3. Assert **strt_cal** for 1 clk cycle
 4. Wait for **cal_done** to be asserted (timeout loop might have to be ~1million cycles)
 5. Let it run for 8 million more cycles (will take a while) and plot the results of heading





Testing Inertial Interface

- Your plot should look similar to the one below if **inert_intf** is working properly



- Only one person (per a 2-person team) need to submit.
- Follow instructions [here](#) to add the 2nd person to the submission on gradescope.

Submit:

- `inert_intf.sv`
- `inert_intf_tb.sv`
- `waveform`